

Assignment Package 4 – Lab2

General instructions: How and when to submit and what to submit are explained on the submission page on Blackboard. This document describes the assignments.

This information is available as part of the course description available in Blackboard:

- All assignment packages are divided into two parts called **Part 1** and **Part 2**. Part 1 is done to get a pass on the exam, while Part 2 is done to get bonus points for a higher grade (4, 5) on the second part of the exam (see the information below about the exam structure). That's it, Part 1 must be done to get a pass, while Part 2 is only done to collect bonus points for a higher grade on the exam.
- The total number of bonus points you can get by solving the tasks on Part 2 for each assignment package is **10**. That's it, you can collect half of the points for a higher grade on the exam by solving these Part 2 tasks. The bonus points you get for the exam are equal to the sum of the bonus points you have received from the solutions to the Part 2 tasks.

Assignments - part 1

1. General description (the subtasks below explain what you should do)

In this assignment, you will implement a hash table version of a Map collection using a variant of hash tables that differs from the one used in the lecture. The API for the collection is the same as for `MapHashSeparateChaining<T1, T2>` (except for the class name).

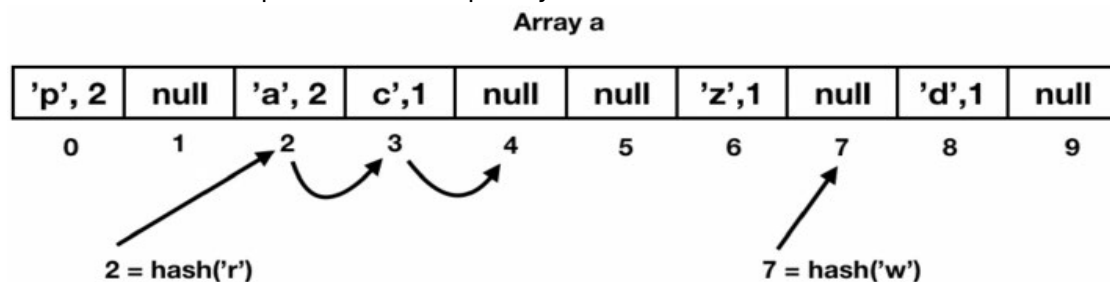
In this variant of the hash table, we use a closed table and linear probing to find free positions to insert keys when collisions occur. We call the collection to be implemented **LinearProbingHashMap<T1, T2>**.

To implement a Map data type using the linear probing technique, we can proceed as follows. We have an array *a* of length *M* in which we store keys. The position where a key is stored is determined both by the hash value of the key and by whether another key is already stored at that position. When inserting a key, one first computes its hash value, which we denote as **hashValue**. If **a[hashValue]** is null, we set **a[hashValue] = key**. Otherwise, we search the hash table starting from position **hashValue** until we find the first **null** position and place the key there.

To store the values associated with the keys, there are two possible approaches:

- storing pairs containing both key and value (using the Pair class) in the array *a*:
Pair<T1, T2>[] a
- or using two arrays of equal length: one for keys and one for values. The value associated with a key is stored at the same index in its array as the key in its array. Instead of one field *a*, you have two fields: **a_keys** and **a_values**:
T1[] a_keys
T2[] a_values

Insertion - The process for insertion is illustrated below with an example where **T1** is of type `Character`. At each position that is not null, there is a pair of type **Pair<Character, Integer>**, where the number represents the frequency of the character.



Deletion - When removing a key (delete), one first computes its hash value **hashValue** using the function **hash()** (you should use the same one presented in the lecture). Then, one iterates from position **hashValue** to the first position in the array where the pair has the key (if it exists) and sets that element to null. After that, the subsequent elements in the array must be repositioned. This is done by iterating through the array from the next position until a **null** value is encountered. In each iteration, one removes the element at the current position (sets it to null and decreases the number of elements by 1) and reinserts it using the insertion method (which in turn increases the number of elements).

The size - It is important that there are not too many elements stored in the array relative to the number of available positions. To ensure this does not happen, the array size is **doubled** when necessary. If the array size is **M** and the number of elements is **N**, then **loadFactor = (double)N/M** represents the proportion of stored elements relative to the total capacity. If the loadFactor is greater than or equal to 0.5, **the array is doubled during insertion**. If the loadFactor is less than or equal to 1.0/8, **the array is halved during deletion**. If you prefer a less dense array, you may use a lower threshold value for the load factor.

Iterating with an order? We want to be able to iterate over the data structure ordered by **T2** (the values, not the keys), with the largest values first, via a method `keys(Comparator<T2> cmp)` (note that this method has been added to the API). To achieve this, one can insert the pairs from the array into a priority queue and then extract the elements from the queue until it is empty.

a) (8 points): The first task is to implement the following API using a linear probing hash table: (**Obs:** Create the *LinearProbingHashMap.java* file)

class LinearProbingHashMap<T1, T2>

```

public LinearProbingHashMap ()
public LinearProbingHashMap (int M)
public int size()
public boolean isEmpty()
public boolean contains(T1 key)
public T2 get(T1 key)
public void put(T1 key, T2 val)
public void delete(T1 key)
public Iterable<T1> keys()
// The new method explained in the text
// Note that the argument is for comparing the value (T2)
public Iterable<T1> keys(Comparator<T2> cmp)

```

b) (2 points): Write a program **WordFrequency** that, using the implemented class in a), counts the number of occurrences of words in a text file and prints all words along with their frequencies, sorted by frequency. (**Obs:** Create the *WordFrequency.java* file)

For example:

```

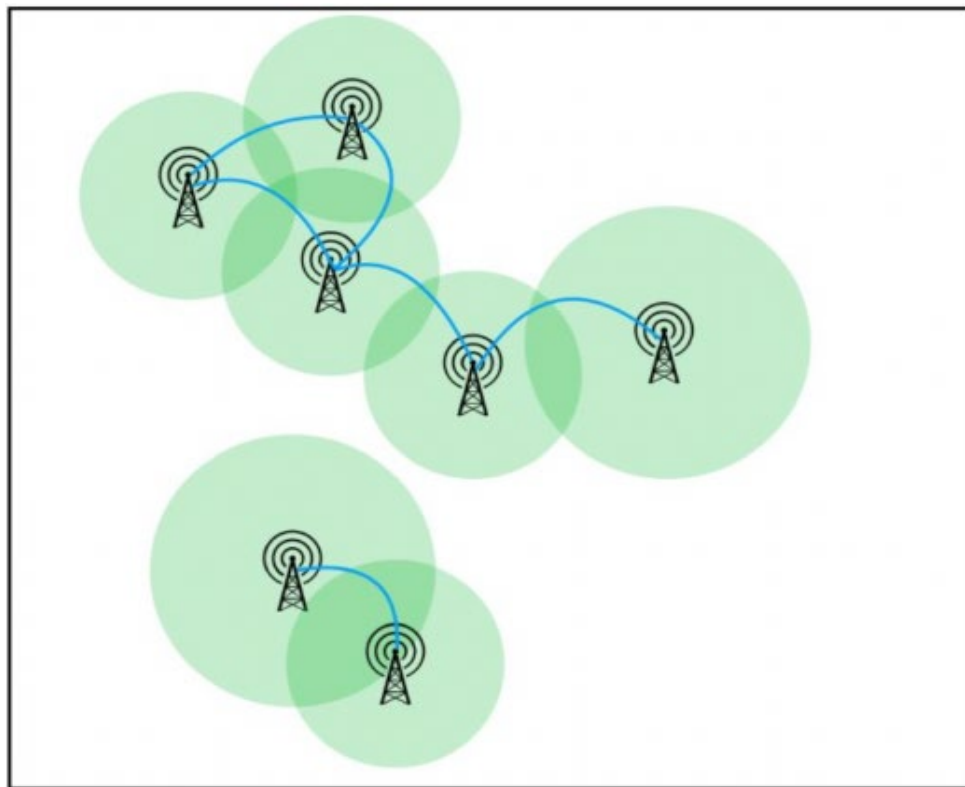
java WordFrequency < text
this 2
is 2
a 1
sentence 1
but 1
not 1

```

if the file contains: "this is a sentence but this is not". Note that words with the same frequency do not need to be sorted by key.

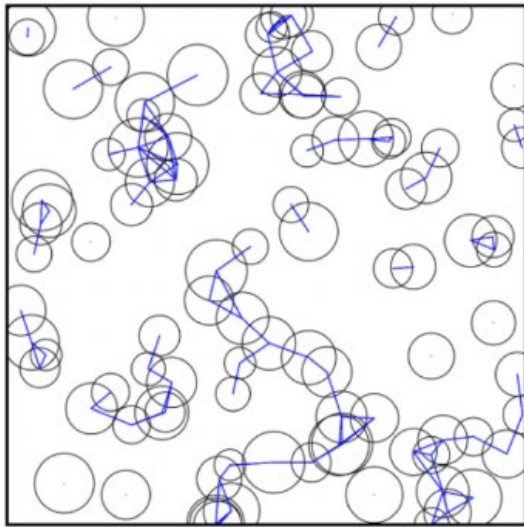
2. Background (the sub-assignments below explain what you should do)

A telecom operator has deployed antennas in a geographical area. A reasonably realistic model of how signal strength decreases with distance from an antenna would at least include the Friis equation, but preferably a more realistic environment-specific model. In what follows, however, we make the simplified assumption that the antennas are omnidirectional and have a threshold distance for acceptable signal strength for communication. This means that around each antenna there is a circular area within which information can be transmitted; outside the circle, communication is not possible. **This is illustrated in the figure below.** We have seven antennas, each with a position and a radius within which it can transmit information. In this example, we see that communication is possible among the five upper antennas and among the two lower antennas, but not between these groups. Information in the upper network of five antennas may sometimes need to pass through other antennas, but they form a connected network of five antennas.

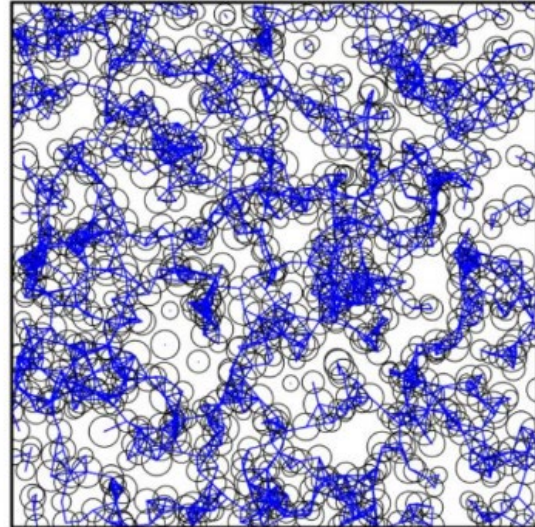


Attached to this assignment package are two text files: **karta1.txt** and **karta2.txt**, which represent maps of geographical areas with antennas. Each antenna is represented as a node with an x-position, a y-position, and a radius indicating how far it can reach. Visualizations of these maps are shown below. You may, for example, use the provided **StdDraw.java** to reproduce them, but you may of course use any other drawing tool of your choice.

In the images, for clarity, **blue lines** are drawn between antennas that are connected (*their circles overlap*).



karta1.txt



karta2.txt

a) (3 points): The first task is to use the **Graph class** to generate a graph from each of the text files **karta1.txt** and **karta2.txt**. Two nodes are neighbors in the graph if their circles overlap; therefore, you must determine which nodes are neighbors in order to construct the graph. This should form the basis of a **class AntennaGraph** (*AntennaGraph.java*).

Once task **a)** is completed, we want to support the following functionality, see **b) – e)**, which builds a graph and has methods to do what is required in the tasks below. In what follows, we use the terms **node** and **antenna** interchangeably.

b) (1 point): It should be possible to determine whether two nodes are connected. The method in the **AntennaGraph class** should have the signature:

public boolean connected(String a, String b)

c) (1 point): It should be possible to determine the number of networks, i.e., the number of connected components. The method should have the signature:

public int nrOfNwks()

d) (2 points): It should be possible, for an arbitrary node, to determine the distance to the node in its connected component that is farthest away. In other words: for any given antenna, compute the **Euclidean distance** to the antenna in its network that is farthest away. The method should have the signature:

public double distanceToBorder(String a)

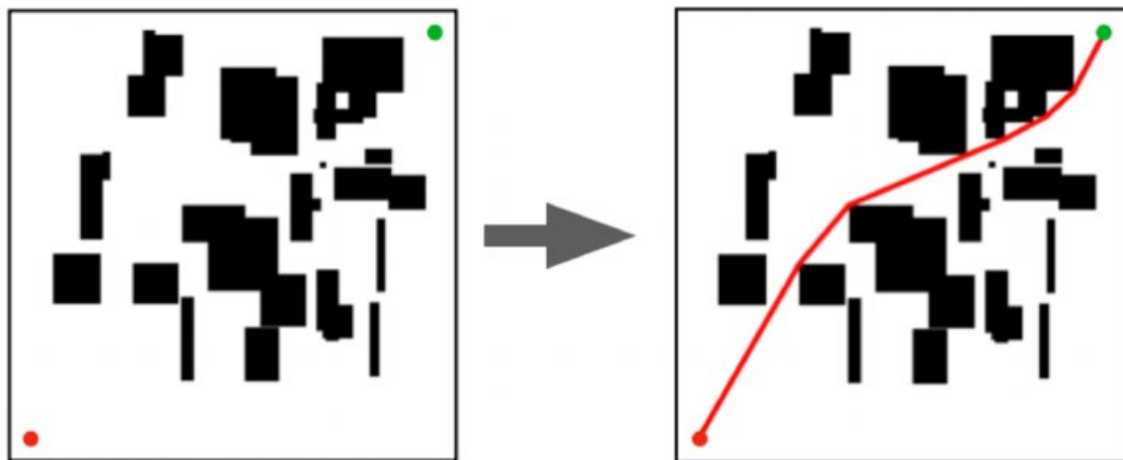
e) (1 point): For two nodes **v** and **w**, one wants to determine the shortest path between them. By shortest path in this context, we mean the shortest sequence of nodes through which one can travel from **v** to **w**. We are not concerned with the physical distance, only with how many antennas must be traversed. The method should have the signature:

public int shortestDistance(String a, String b)

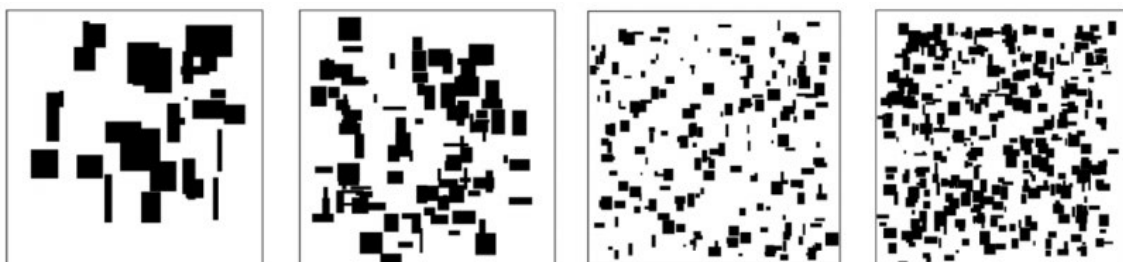
BONUS!!!**Assignments - part 2**

1. There is only one assignment. It is divided into **two parts**: a) and b). Points are specified for each subtask.

General description: Overall, the task is to find (in different ways and with different interpretations) the shortest path between two arbitrary points in two-dimensional maps, where obstacles (buildings) are represented by polygons. See the example below, where we want to find the shortest path between the **red point** and the **green point**.



See examples of four maps below. In all examples, the red point is set to position (50,50) and the green point to (950,950), but it should be possible to choose any two points (as long as they are not inside an obstacle or building).

**Area 1****Area 2****Area 3****Area 4**

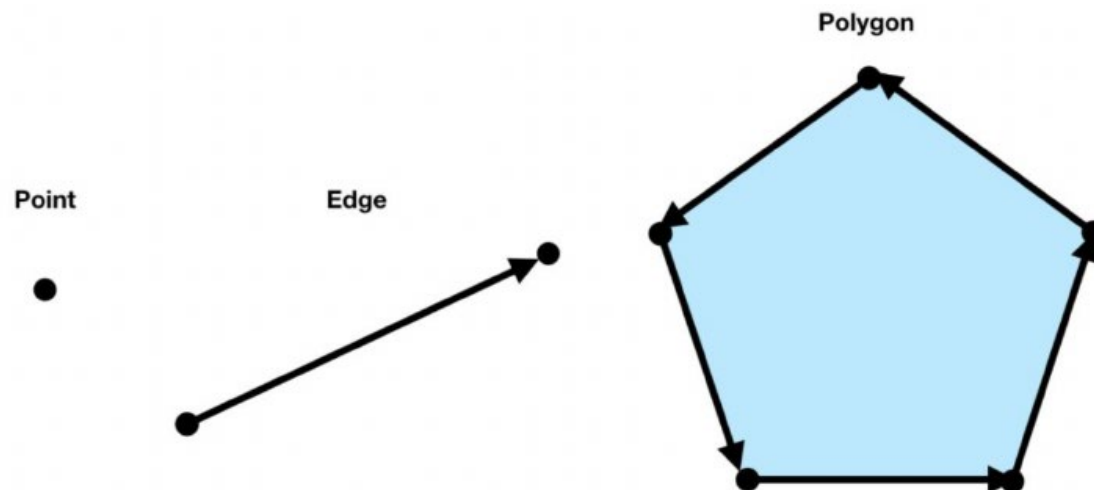
These maps are attached to this problem description as text files. The **first line** of the text file represents the valid boundary of the map. For example, if it states:

Boundary: (0,0) (1000.0,1000.0)

this means that the **lower-left corner** of the map is the point **(0, 0)** and the **upper-right** corner is the point **(1000.0, 1000.0)**. After that follow all polygons (which represent obstacles or buildings). For each polygon, the edges (or sides) are given in the form (x1, y1, x2, y2). Here, (x1, y1) is the first point of the edge and (x2, y2) is the second point. In all these maps, the polygons are rectangles, so there are only four edges (sides).

To visualize these maps, you can use methods from the provided **StdDraw.java**. See the main method in StdDraw for examples of how to draw a polygon. Of course, you may create a more advanced graphical interface. However, the important aspect is that you can visualize things in some way while developing the algorithms.

a) (1 point): In this first task, you should create three classes to represent the three geometric objects: point, edge, and polygon. We call the classes **Point**, **Edge**, and **Polygon**.



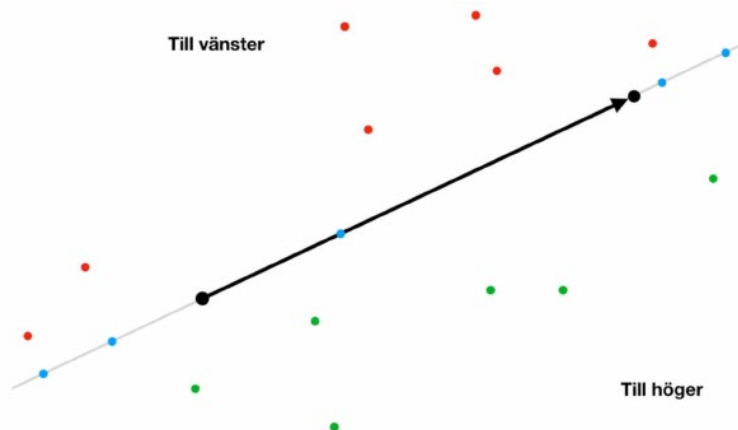
A **point** represents a position in the two-dimensional plane. An **edge** contains two points: a start point and an end point. The **arrow** in the middle image above illustrates the direction from starting point to ending point. The **polygons** we consider are convex and are represented by a circular sequence of edges in **counterclockwise** order, see above.

Obs: For this task create the **Geometry.java** file with all these classes inside, then we have a main class **Geometry**.

To determine whether a point lies on an edge, whether an edge intersects a polygon, whether a point lies inside a polygon, etc., we can introduce the following method in the Edge class:

```
public int compareTo(Point point)
```

which returns a **positive value** if the point lies to the left of the edge, **zero** if the point lies on the line of the edge, and a **negative value** if the point lies to the right of the edge. See the image on the next page for an illustration of the concepts. **Red dots** are to the left, **blue** are on the line the edge lies on, and **green dots** are to the right.



A convenient way to implement the **compareTo** method, without using geometric libraries, is to consider the z-component of the **cross product** between the direction vector of the edge and the direction vector between the starting point of the edge and the point point.

If the edge is (x_1, y_1, x_2, y_2) and the point is (x_3, y_3) , then we calculate the compare value to be returned like this:

```
double dx1 = x2 - x1;
double dy1 = y2 - y1;
double dx2 = x3 - x1;
double dy2 = y3 - y1;
double compare = dx1*dy2 - dy1*dx2;
```

The **compareTo** method can now be used to determine the following:

i. **Whether an edge does not intersect another edge.** This only happens if:

- a) one of the points for one of the two edges lies on the line for the other edge (is a **blue point** see above). Or
- b) for one of the edges both its points lie to the left of the other edge. Or
- c) for one of the edges both its points lie to the right of the other edge.

ii. **Whether a point lies inside a polygon.** This only happens if the point is **to the left** of all the edges that are sides of the polygon.

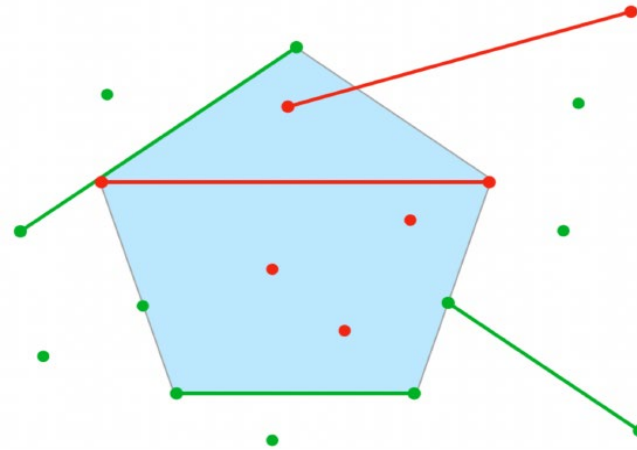
iii. **Whether an edge intersects a polygon.** Here one might think that it is only a matter of going through all edges that are sides of the polygon and for each such side checking whether the edge intersects the side. Then check that none of the **two points on the edge** lie inside the polygon.

However, we see, in the image below, that this would result in an **edge passing through the polygon being allowed**, see the **horizontal red edge** through the polygon. So, in addition to the above, one can also examine these cases. We propose two alternative methods.

Method 1: Create temporary edges between non-consecutive vertices of the polygon. **If the two points of one of these edges are the same as the points of the edge we are examining**, then the edge we are examining **intersects** the polygon.

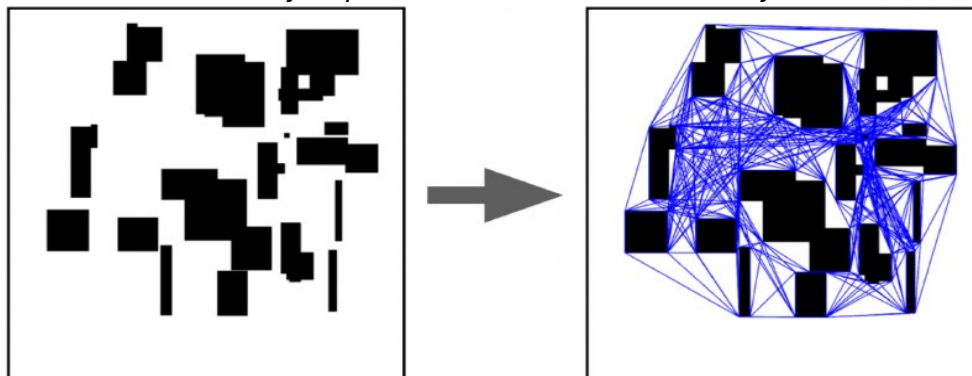
Method 2: Create temporary edges between non-consecutive vertices of the polygon. For each such temporary edge, **find out if this edge is part of the edge we are examining** (lies in the edge we are examining). In such cases, the edge we are examining intersects the polygon.

In practice, we do not need to do **Method 2**, but it is enough to implement **Method 1**. The additional cases investigated in **Method 2** occur with probability of 0 for random polygons. The image below explains much better than this text what to investigate. **Red edges and points** intersect or are **inside the polygon**. For **green ones**, this is not the case.



Finally, once this is completed, one can create a **class Area (Area.java)** that represents a map. Have all the polygons and their sizes been found on the map, etc.

b) (1 point): Now you will create a so-called **visibility graph** where the nodes correspond to vertices of the polygon (*obstacles or buildings*) **that are not in other polygons**. The edges for each node are the other vertices that can be seen from the corner (visibility). What is meant is that you create only those edges that do not intersect any other vertices. Example for Area 1 below. The *VisibilityGraph* class should use the *Geometry Class*.



You should be able to **add points** to an existing visibility graph and expand it. In the example below, we have added the **red** and **green** points.

